

SymPy Bug Report

1 Bug 24: solveset returns 1 for $\operatorname{asec}(x) = 2\pi$ outside the principal asec range

1.1 Minimal Reproducer

```
from sympy import Eq, S, asec, pi, simplify, solveset, symbols, N

x = symbols("x")
eq = Eq(asec(x), 2*pi)
sol = solveset(eq, x, domain=S.Complexes)

print("solution:", sol)
print("residual at returned value:", simplify((eq.lhs - eq.rhs).subs(x, 1)))
print("numeric residual:", N((eq.lhs - eq.rhs).subs(x, 1), 50))
```

1.2 Observed Behavior

```
solution: {1}
residual at returned value: -2*pi
numeric residual: -6.2831853071795864769252867665590057683943387987502
```

SymPy returns the finite solution set $\{1\}$. Substituting the returned value into the original equation does not give zero residual, so the returned point is not a solution of the equation that was solved.

1.3 Expected Behavior

The expected result is $\emptyset$, represented by SymPy as `EmptySet`. The equation

$$\operatorname{asec}(x) = 2\pi$$

has no solution at $x = 1$, and the returned singleton $\{1\}$ should not be reported as the solution set.

1.4 Mathematical Explanation

SymPy's principal inverse secant satisfies

$$\operatorname{asec}(z) = \arccos\left(\frac{1}{z}\right)$$

on its principal branch. In particular,

$$\operatorname{asec}(1) = \arccos(1) = 0.$$

Therefore substituting SymPy's returned value gives

$$\operatorname{asec}(1) - 2\pi = -2\pi \neq 0.$$

The incorrect answer is not an equivalent branch representation of the same solution: asec is a principal inverse function, so $\operatorname{asec}(1)$ denotes the principal value 0, not 2π . Applying $\sec$ to both sides of the equation is therefore not a reversible operation unless the principal-range condition is preserved or the resulting candidate is checked against the original equation.

1.5 Source-Level Diagnosis

The diagnosis locates the responsible behavior in `sympy/solvers/solveset.py`, specifically in the generic inversion logic of `_invert_complex` around lines 550–557. The traced call path is:

```
_invert_complex(asec(_C) - 2*pi, {0})
→ _invert_complex(asec(_C), {2*pi})
→ _invert_complex(_C, {1}).
```

According to the diagnosis, this path does not use the branch-aware `_invert_trig_hyp_complex` helper. Instead, `_invert_complex` uses the generic one-argument `.inverse()` path for `asec`. For `asec(_C)`, the inverse method defined in `sympy/functions/elementary/trigonometric.py` returns `sec`. The solver consequently rewrites

$$\operatorname{asec}(_C) = 2\pi \implies _C = \sec(2\pi) = 1,$$

without checking that 2π is in the principal range of asec , and without rejecting the finite candidate after substitution into the original equation.

A plausible fix direction supported by the diagnosis is to avoid treating inverse trigonometric functions as globally invertible through the generic `.inverse()` path. The implementation could route inverse trigonometric functions through branch-aware handling, add explicit principal-range checks, or verify finite candidates against the original equation before returning them.

```
# Diagnosed transformation:
# asec(_C) = 2*pi
# _C = asec.inverse()(2*pi) = sec(2*pi) = 1
```

1.6 Additional Failing Instances

The independently checked family uses targets of the form $\operatorname{asec}(x) = 2\pi n$. For every listed nonzero integer n , SymPy reports `{1}`, but the same principal-value check applies:

$$\operatorname{asec}(1) - 2\pi n = -2\pi n \neq 0.$$

Thus the expected behavior is to avoid returning `{1}`; for these cases, the independently recorded oracle using $\operatorname{asec}(z) = \arccos(1/z)$ also rejects the returned value.

Input / Expression	SymPy behavior	Expected behavior
$\text{asec}(x) = 2\pi$	returns $\{1\}$; residual at 1 is -2π	must not return $\{1\}$
$\text{asec}(x) = 4\pi$	returns $\{1\}$; residual at 1 is -4π	must not return $\{1\}$
$\text{asec}(x) = -2\pi$	returns $\{1\}$; residual at 1 is 2π	must not return $\{1\}$
$\text{asec}(x) = 6\pi$	returns $\{1\}$; residual at 1 is -6π	must not return $\{1\}$
$\text{asec}(x) = 8\pi$	returns $\{1\}$; residual at 1 is -8π	must not return $\{1\}$
$\text{asec}(x) = 10\pi$	returns $\{1\}$; residual at 1 is -10π	must not return $\{1\}$

1.7 Related Issues Assessment

The best-effort deduplication check found related public material documenting the principal behavior of asec , including that $\text{asec}(1) = 0$, and general context about branch-sensitive inverse trigonometric functions. It did not identify a public SymPy issue, pull request, or discussion specifically reporting that `solveset(Eq(asec(x), 2*pi), x, domain=S.Complexes)` returns $\{1\}$. The deduplication verdict is therefore a new instance of a known failure mode: global inversion of principal inverse trigonometric functions can produce extraneous solutions, but this exact minimized asec case is previously unreported in the available evidence and remains individually actionable as an issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Eq, FiniteSet, S, asec, pi, simplify, solveset, symbols

@pytest.mark.parametrize("n", [1, 2, -1, 3, 4, 5])
def test_solveset_asec_rejects_out_of_range_targets(n):
    x = symbols("x")
    eq = Eq(asec(x), 2*pi*n)
    sol = solveset(eq, x, domain=S.Complexes)

    assert sol != FiniteSet(1)
    assert simplify((eq.lhs - eq.rhs).subs(x, 1)) != 0
```